

实验 1

阿里云百炼平台 API 调用

殷悦扬

2023级管院 统计学

录制：线下课；线上课



复旦大学 管理学院
SCHOOL OF MANAGEMENT
FUDAN UNIVERSITY

开智求真 拓新领变

Enlightening for Truth Pioneering for Change



- **主要内容：**

- 掌握 Qwen 系列大模型调用方法，能够实现多轮对话、流式输出、结构化输出等
- 了解如何将图像（本地 / URL）、文件通过 API 上传给大模型
- 了解上下文缓存、批量推理（省钱）、多线程并行调用（省时）
- 掌握实际应用中的异常处理方法（案例演示）

- **课后作业：**

- 酒店评论数据清洗与特征提取
- 调用大模型 API 完成评论质量评估、类别划分、摘要生成、反向 Query 生成
- 选做：模型一致性评估
- 思考题 1

- **场景：调用大模型处理 10,000 条酒店评论（反向 Query 生成）**
 - 系统提示词：约 200 tokens（每次请求都要重复发送）
 - 每条评论输入：约 100 tokens，输出：约 50 tokens
 - 模型：Qwen-Plus（输入 0.8 元 / 百万 Token，输出 4.8 元 / 百万 Token）
- **直接调用的成本：**
 - 总输入 = $10,000 \times (200 + 100) = 300$ 万 tokens
 - 总输出 = $10,000 \times 50 = 50$ 万 tokens
 - 费用 = $3.0 \times 0.8 + 0.5 \times 4.8 = 4.8$ 元
- **其中：200 tokens 的系统提示词被重复发送了 10,000 次，占输入的 67%**
- **如何省钱？** 上下文缓存（减少重复输入费用） / 批量推理（全部半价）

- **什么是上下文缓存？与 KV Cache 的关系是什么？**

- Transformer 生成每个新 token 时，需要对前面所有 token 做注意力计算，KV Cache 把已生成 token 的 Key/Value 向量缓存起来，下一个 token 直接复用，不重复计算——这是**单次请求**内部的优化，所有推理引擎都会默认开启
- 上下文缓存则更进一步，把 KV Cache **跨请求共享**：多个请求如果有相同前缀，后续请求直接复用之前算好的 KV 状态，省掉重复的 prefill 计算

- **不同缓存模式**

- **隐式缓存**：系统自动识别重复前缀并缓存，无需额外代码，默认开启
 - 命中缓存的输入按标准价格的 **20%** 计费（百炼平台）
- 还可以主动为指定内容创建**显式缓存**、在多轮对话中使用 **Session 缓存**
 - 创建缓存按输入价格的 125% 计费，后续命中仅 **10%**（百炼平台）

- 命中机制：前缀匹配

- 系统缓存了 "ABCD", 请求 "ABE" 可命中 "AB", 但请求 "BCD" 无法命中
- 隐式缓存无法保证 100% 命中, 长时间未使用的缓存会被清理 (并发请求更有利于命中缓存)

- 应用方法与典型场景

- 将重复内容放在提示词开头 (如系统提示词), 差异内容放在末尾 (如用户提示词)
- 多模态理解: 对同一图片多次提问时, 图片放在文字前面
- 多轮对话: 每轮请求的前缀与上轮相同
- 代码补全: 不断增长的代码上下文

- 相关语录 (Manus 联合创始人)

- “如果非要我只选一个指标, 我认为 **KV 缓存命中率** 是生产级 AI Agent 最重要的单一指标, 因为它直接影响**延迟和成本**。”

• 案例讨论

– 如何评价以下做法？

场景描述

一位工程师希望优化 Agent（如 Coding Agent）的上下文

正常 Agent 提示词结构（简化版）：

系统提示词 → 工具列表 → 用户提示词 → 模型回复 → 用户提示词 → 模型回复 → ...

已知问题：

上下文中工具过多会对模型性能造成负面影响（增加干扰、降低准确率）

优化方案

每轮对话都根据用户提示词的任务类型，动态过滤工具列表：删除 Agent 上下文中当前任务一定不需要的工具，减少工具数量（每轮都从完整工具列表中进行过滤）



• 案例讨论

– Claude Opus 4.6 Extended 回复：

这是一个在实践中相当常见且合理的工程优化思路，但它既有明显的好处，也有需要注意的陷阱。

为什么这个做法有道理

大量工具定义确实会挤占上下文窗口、稀释模型对每个工具的注意力，并增加模型"选错工具"或"幻觉调用"的概率。动态裁剪工具集，本质上是在帮模型缩小决策空间，让它更专注——这和人类在面对选择时"减少无关选项反而做得更好"是一个道理。

做得好的话，收益很实在：

准确率提升（更少的工具混淆和误调用）、延迟降低（更短的 prompt 意味着更快的推理）、成本节约（token 用量下降），这些在工具数量多到几十甚至上百时尤其显著。

但关键在于"过滤器"本身的设计质量

但它是不是破坏了上下文缓存？

滤与缓存命中率的工程取舍。 >

这一针见血，点到了这个方案最大的工程代价。

确实会破坏上下文缓存，而且代价可能很大。

Anthropic 的 prompt caching（以及其他提供商的类似机制）是基于 prompt 前缀的精确匹配来工作的。系统提示词和工具定义通常排在最前面，恰恰是前缀的一部分。每轮动态增删工具，前缀就变了，缓存直接失效——哪怕只少了一个工具，整个缓存都命不中。

这意味着每一轮对话都要重新处理完整的 prompt，丢掉的是缓存带来的延迟降低和成本折扣（缓存命中时 token 费用通常是原价的 10% 左右）。对话越长、工具定义越多，这个损失越大。

- **什么是批量推理？**

- 对于无需实时响应的场景，将大量请求打包提交，由系统异步处理，费用仅为实时推理的 **50%**

- **批量调用方式**

- 通过百炼平台上传对应格式的 JSONL 文件，系统后台处理，完成后下载结果

- **适用场景**

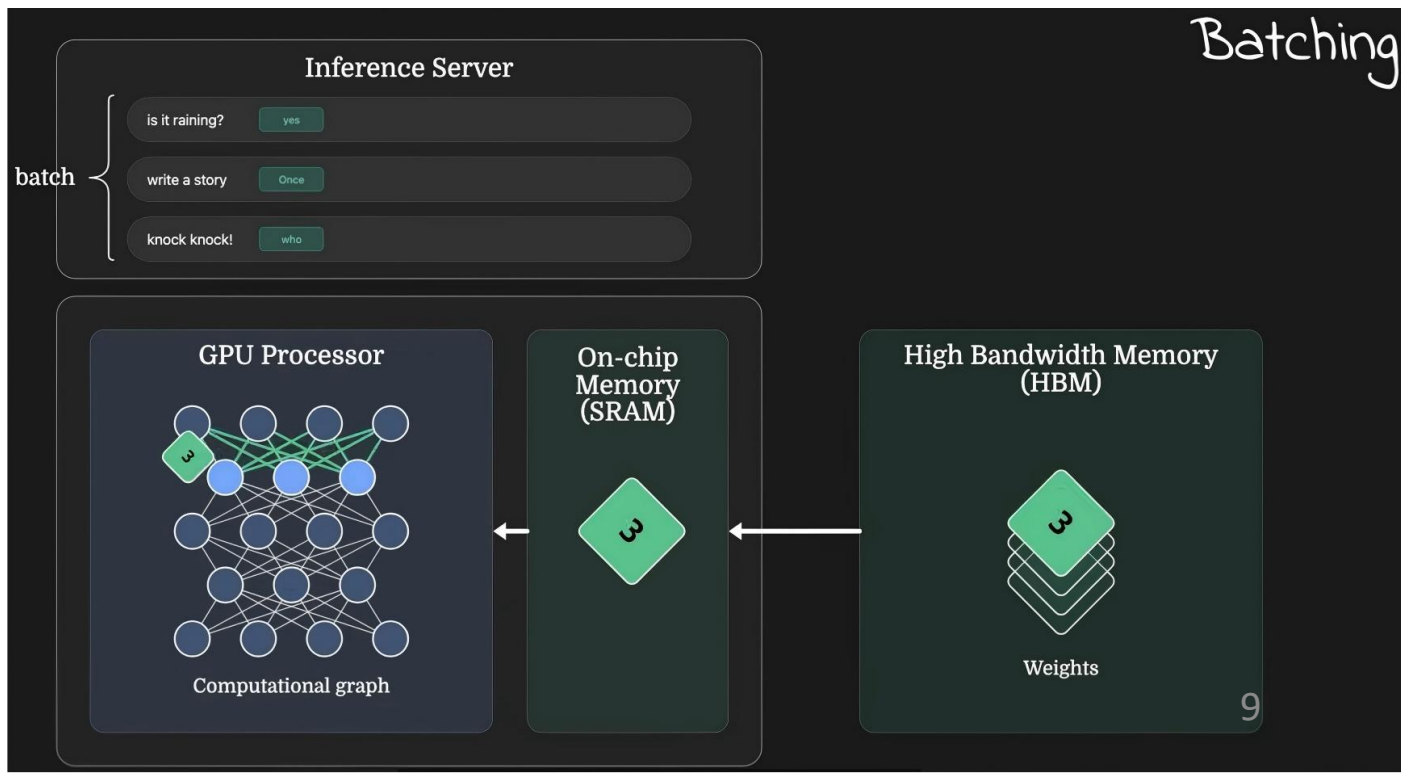
- 数据标注、内容生成、模型评测等大批量作业
- 对时效性要求不高，但对成本敏感的任务

- **注意事项**

- 批量推理不支持上下文缓存，也无法使用新人免费额度优惠
- 执行时间取决于系统资源调度，非实时返回

为什么批量推理能节省成本？

- Decode 阶段瓶颈不在 GPU 算力而在**带宽限制**，每次前向传播都需要从 HBM 读取权重到 SRAM，而 Batching 让每一次权重读取同时为多个请求计算 next token，摊薄了单个 token 的带宽开销
- Continuous Batching：每次迭代后都会重新配置批处理，及时剥离已完成回复的请求
- 思考：Prefill 阶段的瓶颈是什么？
- 其他原因：峰谷调度等（类似电网）





- **回到开头的例子：10,000 条评论，Qwen-Plus**
 - 直接调用：4.8 元
 - 上下文缓存若全部命中可节省： $2.4 \text{ (输入费用)} \times \frac{2}{3} \times 80\% = 1.28 \text{ 元}$ ，约 **27%**
 - 批量推理：全部半价，省 **50%**
- **在实验中的应用：**
 - 检查提示词是否能够充分命中缓存
 - 利用多线程并发请求提升缓存命中率并节约时间（代码中讲解）
 - 尝试一次批量推理



復旦大學 管理学院
SCHOOL OF MANAGEMENT
FUDAN UNIVERSITY

开智求真 拓新领变
Enlightening for Truth Pioneering for Change



2026-3-24



11